COMP 3004 Winter 2026

HintonMarket Deliverable 1

Due: Thursday, March 12, 2026, at 23h59

Collaboration: This deliverable must be completed by registered teams

---

Overview

Your team will design and build the first working version of the **Hintonville Farmers Market Management System (HintonMarket)**.

This deliverable focuses on defining a **system decomposition diagram** and producing a **functional in-memory vertical prototype** that demonstrates core browsing of available market dates, booking a stall, joining a market date waitlist, and cancelling a stall booking for the Vendor.

This deliverable builds upon the *HintonMarket Project Overview* document.
No database or file persistence is required for D1; your system operates **entirely** in memory.

Part 1 – System Decomposition Diagram (20 marks)

Submission must include the following in a PDF document:

1. **One** UML class diagram showing a system decomposition. The diagram must be embedded in the PDF document.
2. Written explanation about how your system design minimizes coupling and maximizes cohesion in the system your team have implemented.

Requirements

- You will provide one UML class diagram showing the subsystem decomposition of a subset of the HintonMarket system that your team implemented. Refer to Part 2 for the use cases your team will implement.
- Your diagram must show all entity, control, and boundary classes:
    - You must show only class names; do not show attributes or operations.
    - The entity objects must be relevant to the chosen detailed use cases that you implemented in your functional implementation.
    - The control and boundary objects must be the ones associated with the use cases implemented.
- Your diagram must show all associations between classes:
    - It is insufficient to show the associations between subsystems; you must show the associations between the individual classes.
    - Classes must be grouped into subsystems, where subsystems are shown as UML packages of classes; each subsystem must be labelled with its name.
- Class groupings into subsystems must minimize coupling and maximize cohesion.
- Your **document** must explain, in your own words, how your design minimizes coupling and maximizes cohesion.
- Reference the MyTrip example subsystem decomposition (Class Notes Section 3.3.3, Figure 6.29) as a model for organizing and presenting your subsystems.

Submission for Part 1

Include the system decomposition diagram in a PDF document. You must include a descriptive figure title for your diagram (e.g., "Figure 1: HintonMarket System Decomposition Showing Subsystem Organization and Dependencies") and your explanation about how your design minimizes coupling and maximizes cohesion. Note: Your diagram must be embedded in the PDF document (not a link to a diagram).

Part 2 – HintonMarket Vertical Prototype (80 marks)

Default System Data

At startup, your program must load:

- **MarketSchedule** must include all of the following:
  - 4 weeks of markets are available for vendors to book.
  - A maximum of 2 food vendors and 2 artisan vendors market stalls are available per week (not the 20 food vendors and 10 artisan vendors mentioned in the HintonMarket overview document). *This will allow for easier testing the waitlist for a market stall.*

- **10 unique users,** including all the following:
  - **4 unique food vendors**
    - With vendor information, compliance documents information, name, and address
  - **4 unique artisan vendors**
    - With vendor information, compliance documents information, name, and address
  - **1 market operator**
  - **1 system administrator**

All data is held in memory only (no database in this deliverable). Note the data must be stored as ***objects*** in memory.

When the program restarts:

- All market dates will have 0 stalls booked.
- The market stall waitlist will be reset to empty.
- All vendors' stall bookings reset to zero.

Functional Requirements

Implement the following functionality for **Vendor validating into HintonMarket and activity**:

1. **Vendor Identification (Not True Authentication)**
   - Users enter their name or username on the startup form; the system verifies the name against the in-memory data structure and displays the appropriate interface based on their user type (Vendor, Market Operator, or System Administrator).
   - Exiting the system returns the user to the startup form and clears the previous user's data to prevent information from carrying over between users.

Vendor Features (Must Implement)

2. **Browse Available Market Dates:**
    - View all available market dates, only four weeks**.**
    - Display
        i. Stall availability **by vendor category** (Food vs Artisan) for each date
        ii. Current booking status
3. **Book a Stall**
    - Confirmation message displayed upon successful booking.
    - Booking appears on the vendor's status dashboard.
    - Enforce a maximum of 2 artisan or food vendors per market day.
    - Update stall booking status in the market schedule.
    - Add to the vendor's booked stall.
4. **Cancel Stall Booking**
    - Confirmation message displayed upon successful cancellation.
    - Booking removed from the vendor's status dashboard.
    - Update stall booking status in the market schedule.
    - Remove from the vendor's booked stalls.
5. **Place on Waitlist**
    - Allow the vendor to be placed in the vendor type (artisan or food vendor) waitlist queue when no stalls are available for their category on a specific market day.
    - Add vendor to the category waitlist queue for market week.
    - Display queue position in the vendor's status dashboard.
6. **Cancel Waitlist placement**
    - Remove vendor from waitlist queue.
    - Update waitlist queue positions for remaining vendors in that category for that market day.
    - Notify the next vendor in the waitlist queue that they can book the stall.
7. **View Vendor Status Dashboard**

    Implement a dashboard view accessible to vendors showing:
    - Business information
        i. Business name
        ii. Owner name
        iii. Contact information (email, phone, mailing address)
        iv. Vendor category (Food or Artisan)
    - Compliance Documentation Status
        i. License and policy details with document numbers (refer to HintonMarket overview for types of licences and policy details)
        ii. Expiration dates for all compliance documents
    - Display active stall booking with:
        i. Market date

- Display active waitlist with:
    i. Market date
    ii. Queue position
- System notifications
    i. Alerts when the vendor is next in line on the waitlist and a stall is available
    ii. Confirmation messages for completed stall bookings
    iii. Confirmation messages for stall cancellations

Business Rules

- Waitlists are organized by vendor category (Food or Artisan) AND by market week
- Maximum of 8 active waitlists (4 weeks × 2 categories)
- Each waitlist follows first-come, first-served (FIFO) order
- When a stall becomes available via cancellation, the system notifies the first vendor in the appropriate waitlist queue (matching both category and week)
- Vendors can book only one market stall date at a time.

Excluded from this Prototype:

- No stall payment processing
- No booking deadline enforcement (Wednesday 11:59 PM booking window)
- No cancellation deadline enforcement (Wednesday 12:00 PM cancellation deadline)
- No refund processing

Grading Breakdown

Part 1- System Decomposition Diagram 20 marks
Part 2 - Functional Implementation 80 marks

Implementation Requirements

(This section defines the technical environment, design standards, and quality attributes that constrain how the HintonMarket system must be built.)

- The following requirements specify the **technical environment, design expectations, and functional behaviour** of the HintonMarket system. Your implementation must meet these standards to ensure it operates correctly within the course VM and demonstrates appropriate object-oriented design.
- The Linux Ubuntu platform, as provided in the official course virtual machine (VM), will be used as the test bed for evaluating your code.
- All source code must be written in C++, and it must be designed and implemented at the level of a 3rd-year undergraduate Computer Science student, following standard UNIX programming conventions, and using advanced OO programming techniques such as polymorphism and design patterns.
- Only libraries already provided as part of the course VM may be used.
- The HintonMarket must provide a graphical user interface built with Qt widgets. In general, user features should be easily navigable, either as menu items and/or pop-up menus. The look-and-feel of the HintonMarket system should be professional and consistent with commercially available UIs.
- The HintonMarket system will run on a single host.
- Data storage for Deliverable 1 will be in memory.

Submission Requirements

(This section defines how your project must be packaged, extracted, built, and run for grading.)

- Your submission must be fully turnkey; the TA must be able to extract the zip file, then import, build, and run your application in Qt Creator on the course VM in under three (3) minutes

.**Requirements:**

- The TA must be able to **extract your ZIP file** and open the project directly in **Qt Creator**.
- The TA must be able to **build and run the application within Qt Creator** without changing any file paths, settings, or configuration options.
- **All required files** (source code, .pro, resources, and data) must be included in your zip file. The TAs will open your project in Qt Creator, loading the .pro. If no .pro file is available, then the TAs will not be able to load and run your project, and you will receive 0 marks for the functional implementation.
- **No manual installation of libraries**, environment setup, or directory relocation is permitted.
- All setup and usage instructions, as well as the **application vendor user names**, must be **clearly documented** in your README file.
- The project must compile and execute correctly **as described under Implementation Requirements**.
- Projects that do not open, build, and run in Qt Creator as described **will not be graded**.

Format Requirements

- File Naming and Format: Submit as PDF using naming convention team_#_D1.pdf (# is your group number in BrightSpace). Documents must be typed, legible, and professional, including a cover page (with title, date, team number and team member names, and student numbers).
- UML Diagrams: All UML diagrams must be created using professional drawing tools. Hand-drawn diagrams will not be accepted. Note that the diagram must be embedded in the PDF document and **not** as a link. If a link to a diagram is given in the PDF document, then you will receive 0 for the System Decomposition Diagram and explanation.
- Documents that do not conform to these specifications will earn a grade of zero.
- Coding deliverables must be provided as a single zip file containing all source code and data files, as well as installation, build, and launch instructions, and names of user accounts in a README file and a system decomposition diagram PDF. Do not provide object files and project executables as part of your submission. Zip file naming convention: team_#_D1.zip.
- Your coding deliverable must be submitted as a single zip file (team_#_D1.zip, where # is your team number) containing:
    - All source code and data files
    - Qt project file (.pro)
    - README file with:
        - Installation, build, and launch instructions
        - User account names for testing applicatioin
    - System decomposition diagram (PDF) and explanation
    - **Do not include:** object files or compiled executables in your submission.